

Estructuras de Control en Arduino

¿Qué son las estructuras de control?

Las estructuras de control permiten que tu programa **tome decisiones** y **repita código**. Sin ellas, tu programa solo ejecutaría cada línea una vez, en orden.

Con estructuras de control puedes:

- **Decidir** qué código ejecutar según condiciones
 - **Repetir** código múltiples veces
 - **Controlar el flujo** de ejecución
-

PARTE 1: CONDICIONALES

1.1 if (Si)

Ejecuta código **solo si** una condición es verdadera.

Sintaxis:

```
if (condicion) {  
    // Código si es true  
}
```

Ejemplo:

```
int edad = 20;  
  
if (edad >= 18) {  
    Serial.println("Eres mayor de edad");  
}
```

Flujo:

```
¿edad >= 18?  
→ SÍ → ejecuta el código  
→ NO → salta el bloque
```

Importante: Si la condición es false, el bloque se ignora completamente.

1.2 if-else (Si-sino)

Ejecuta un bloque si la condición es verdadera, y otro bloque si es falsa.

Sintaxis:

```
if (condicion) {  
    // Código si es true  
} else {  
    // Código si es false  
}
```

Ejemplo:

```
int edad = 15;

if (edad >= 18) {
    Serial.println("Eres mayor de edad");
} else {
    Serial.println("Eres menor de edad");
}
```

Salida:

Eres menor de edad

Flujo:

```
¿edad >= 18?
→ SÍ → ejecuta if
→ NO → ejecuta else
```

Importante: Siempre se ejecuta UNO de los dos bloques, nunca ambos.

1.3 if-else if-else (Si-sino si-sino)

Permite evaluar **múltiples condiciones** en secuencia.

Sintaxis:

```
if (condicion1) {
    // Si condicion1 es true
} else if (condicion2) {
    // Si condicion1 es false y condicion2 es true
} else if (condicion3) {
    // Si condicion1 y condicion2 son false y condicion3 es true
} else {
    // Si ninguna condición anterior fue true
}
```

Ejemplo:

```
int nota = 75;

if (nota >= 90) {
    Serial.println("Excelente");
} else if (nota >= 80) {
    Serial.println("Muy bien");
} else if (nota >= 70) {
    Serial.println("Bien");
} else if (nota >= 60) {
    Serial.println("Aprobado");
} else {
    Serial.println("Reprobado");
}
```

Salida:

Bien

Flujo:

```
{nota >= 90? NO
{nota >= 80? NO
{nota >= 70? SÍ → ejecuta "Bien" y termina
```

Importante: Solo se ejecuta el PRIMER bloque que cumpla la condición. Los demás se ignoran.

1.4 switch-case (Seleccionar caso)

Compara una variable contra múltiples valores exactos.

Sintaxis:

```
switch (variable) {
  case valor1:
    // Si variable == valor1
    break;
  case valor2:
    // Si variable == valor2
    break;
  default:
    // Si no coincide con ningún caso
    break;
}
```

Ejemplo:

```
char opcion = 'B';

switch (opcion) {
  case 'A':
    Serial.println("Opción A");
    break;
  case 'B':
    Serial.println("Opción B");
    break;
  case 'C':
    Serial.println("Opción C");
    break;
  default:
    Serial.println("Opción no válida");
    break;
}
```

Salida:

Opción B

Importante:

- **break** es OBLIGATORIO para salir del switch
- Sin break, el código **sigue ejecutando** los casos siguientes (fall-through)
- **default** es opcional

¿Cuándo usar switch vs if-else?

- **switch**: Cuando comparas UNA variable contra MÚLTIPLES valores exactos
 - **if-else**: Cuando necesitas condiciones complejas o rangos
-

PARTE 2: BUCLES

2.1 for (Para)

Repite código un **número específico de veces**.

Sintaxis:

```
for (inicializacion; condicion; incremento) {  
    // Código a repetir  
}
```

Explicación:

- 1º **inicializacion**: Se ejecuta UNA VEZ al inicio (crear contador)
- 2º **condicion**: Se evalúa ANTES de cada iteración (¿continuar?)
- 3º **incremento**: Se ejecuta DESPUÉS de cada iteración (actualizar contador)

Ejemplo:

```
for (int i = 1; i <= 5; i++) {  
    Serial.println(i);  
}
```

Salida:

```
1  
2  
3  
4  
5
```

Flujo:

```
i = 1 → ¿i <= 5? SÍ → imprime 1 → i++ → i = 2  
i = 2 → ¿i <= 5? SÍ → imprime 2 → i++ → i = 3  
i = 3 → ¿i <= 5? SÍ → imprime 3 → i++ → i = 4  
i = 4 → ¿i <= 5? SÍ → imprime 4 → i++ → i = 5  
i = 5 → ¿i <= 5? SÍ → imprime 5 → i++ → i = 6  
i = 6 → ¿i <= 5? NO → sale del bucle
```

Variantes:

```
// Contar hacia atrás
for (int i = 5; i >= 1; i--) {
    Serial.println(i);
}

// Incremento de 2
for (int i = 0; i <= 10; i += 2) {
    Serial.println(i); // 0, 2, 4, 6, 8, 10
}

// Sin inicialización (la variable ya existe)
int x = 0;
for (; x < 5; x++) {
    Serial.println(x);
}
```

2.2 while (Mientras)

Repite código **mientras una condición sea verdadera**.

Sintaxis:

```
while (condicion) {
    // Código a repetir
    // Importante: cambiar la condición, sino bucle infinito
}
```

Ejemplo:

```
int contador = 1;

while (contador <= 5) {
    Serial.println(contador);
    contador++;
}
```

Salida:

```
1
2
3
4
5
```

Flujo:

```
¿contador <= 5? SÍ → imprime 1 → contador++
¿contador <= 5? SÍ → imprime 2 → contador++
¿contador <= 5? SÍ → imprime 3 → contador++
¿contador <= 5? SÍ → imprime 4 → contador++
¿contador <= 5? SÍ → imprime 5 → contador++
¿contador <= 5? NO → sale del bucle
```

Importante:

- La condición se evalúa ANTES de ejecutar el código
- **DEBES cambiar la condición dentro del bucle**, sino es infinito
- Útil cuando **no sabes cuántas iteraciones necesitas**

¿Cuándo usar while vs for?

- **for**: Cuando sabes el número exacto de iteraciones (10 veces)
 - **while**: Cuando la condición es compleja o desconocida (mientras el usuario no escriba "salir")
-

2.3 do-while (Hacer-mientras)

Similar a **while**, pero **garantiza al menos una ejecución**.

Sintaxis:

```
do {  
    // Código a repetir (se ejecuta al menos una vez)  
} while (condicion);
```

Ejemplo:

```
int contador = 1;  
  
do {  
    Serial.println(contador);  
    contador++;  
} while (contador <= 5);
```

Salida:

```
1  
2  
3  
4  
5
```

Diferencia con while:

```
// while: la condición se evalúa ANTES  
while (false) {  
    Serial.println("No se ejecuta"); // No se imprime  
}  
  
// do-while: se ejecuta al menos una vez  
do {  
    Serial.println("Se ejecuta una vez"); // Sí se imprime  
} while (false);
```

Salida:

```
Se ejecuta una vez
```

PARTE 3: CONTROL DE FLUJO EN BUCLES

3.1 break (Romper)

Termina inmediatamente el bucle o switch.

Ejemplo:

```
for (int i = 1; i <= 10; i++) {  
    if (i == 5) {  
        Serial.println("¡Detenido en 5!");  
        break; // Sale del bucle  
    }  
    Serial.println(i);  
}
```

Salida:

```
1  
2  
3  
4  
¡Detenido en 5!
```

Importante: El bucle termina completamente, no continúa.

3.2 continue (Continuar)

Salta al siguiente iteración, omitiendo el código restante de la iteración actual.

Ejemplo:

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {  
        continue; // Salta esta iteración  
    }  
    Serial.println(i);  
}
```

Salida:

```
1  
2  
4  
5
```

Importante: El bucle continúa, pero omite la iteración con i=3.

PARTE 4: BUCLES ANIDADOS

Puedes poner un bucle **dentro de otro bucle**.

Ejemplo: Tabla de multiplicar

```
for (int i = 1; i <= 3; i++) {  
    Serial.print("Tabla del ");  
    Serial.print(i);  
    Serial.println(":");  
  
    for (int j = 1; j <= 3; j++) {  
        Serial.print(i);  
        Serial.print(" * ");  
        Serial.print(j);  
        Serial.print(" = ");  
        Serial.println(i * j);  
    }  
  
    Serial.println("");  
}
```

Salida:

```
Tabla del 1:  
1 * 1 = 1  
1 * 2 = 2  
1 * 3 = 3  
  
Tabla del 2:  
2 * 1 = 2  
2 * 2 = 4  
2 * 3 = 6  
  
Tabla del 3:  
3 * 1 = 3  
3 * 2 = 6  
3 * 3 = 9
```

PARTE 5: ESTRUCTURAS ANIDADAS

Puedes combinar condicionales y bucles.

Ejemplo: Bucle con decisión

```
for (int i = 1; i <= 5; i++) {  
    if (i % 2 == 0) {  
        Serial.print(i);  
        Serial.println(" es par");  
    } else {  
        Serial.print(i);  
        Serial.println(" es impar");  
    }  
}
```


Salida:

1 es impar
2 es par
3 es impar
4 es par
5 es impar

PUNTOS CLAVE A RECORDAR

- 1º **if** → decisión simple
- 2º **if-else** → dos opciones
- 3º **if-else if-else** → múltiples opciones
- 4º **switch-case** → comparar contra múltiples valores exactos
- 5º **for** → repetir un número exacto de veces
- 6º **while** → repetir mientras condición sea true
- 7º **do-while** → al menos una ejecución garantizada
- 8º **break** → sale del bucle/switch
- 9º **continue** → salta a siguiente iteración
- 10º Puedes **anidar** estructuras (bucle dentro de bucle, if dentro de bucle, etc.)